département
Informatique

Université Claude Bernard Lyon 1

Année universitaire	**2025-2026**		
Filière	**Data Science**	**Année**	**M2**
Matière	**Machine Learning 2**		
Enseignant	**Haytham Elghazel**		
Intitulé TD/TP :	**Atelier 1 : Traitement de données Textuelles**		
Contenu	• **Préparation de données textuelles** • **Vectorisation de données textuelles** • **TFIDF, LSA** • **Word2vec** • **Apprentissage sur des données textuelles**		

Dans cet atelier pratique, l'objectif dans cette partie est de faire une étude comparative entre plusieurs algorithmes d'apprentissage supervisé sur différents jeux de données textuelles avec le langage **Python**.

Pour lancer le notebook Python, il faut taper la commande **jupyter notebook** dans votre dossier de travail. Une fenêtre va se lancer dans votre navigateur pour ouvrir l'application Jupyter. Créer un nouveau notebook Python et taper le code suivant dans une nouvelle cellule :

```
import numpy as np
np.set_printoptions(threshold=10000, suppress = True)
import pandas as pd
import warnings
import matplotlib.pyplot as plt
warnings.filterwarnings('ignore')
```

Attention : Une attention particulière sera donnée pendant la notation à la factorisation de votre code *en proposant plusieurs mini-fonctions pour chaque traitement proposé.* Pensez à bien mettre toutes vos *mini-fonctions* dans un fichier utils.py

1. Partie 1 : Analyse de données de commentaires

L'objectif dans cette partie du TP est d'apprendre sur un jeu de données de *commentaires* du fichier **"yelp-text-by-stars.csv"** en suivant les étapes suivantes.

- **Importer** ce jeu de données avec la librairie pandas (**c.f. read_csv**)
- **Analyser** votre jeu de données, essentiellement la **target**. **Modéliser** ensuite le problème d'apprentissage supervisé (classification ou régression) sur ces données.
- **Traiter** vos données textuelles en supprimant les bruits dans les textes et en les normalisant. Vous pouvez vous inspirer par exemple du code par ici[1] (en l'améliorant s'il le faut) si vous utiliserez la librairie **NLTK**. Vous pouvez aussi utiliser d'autres librairies (A vos recherches). N'oubliez pas que cette **étape de pré-traitement (preprocessing)** dépend de vos données et du problème traité. Quelques traitements à faire en partie ou totalement sont (*utilisez ceux qui sont en gras*):
  - **Suppression des ponctuations comme . , ! $( ) * % @**
  - Suppression des Stop words
  - **Transformation de tout le texte en minuscule.**
  - **Tokenisation de vos textes**
  - Racinisation (Stemming) **ou** Lemmatisation (lemmatization) **ou** les deux en fonction de la classe du mot.
- **Séparer** les données en jeu de données d'**apprentissage** et jeu de données de **test**.
- **Proposer** une fonction ***run_models*** permettant de comparer plusieurs modèles d'apprentissage (en

[1] https://www.kdnuggets.com/2018/03/text-data-preprocessing-walkthrough-python.html

fonction de votre modélisation) sur ces données dont une **forêt aléatoire,** un **réseau de neurones à deux couches** et un **k-plus proche voisin** (Bien faire attention à la distance utilisée). Votre évaluation se basera sur un seul score à choisir en fonction de votre modélisation.

- **Proposer** une première **vectorisation** de vos données textuelles par une représentation **bag-of-words**.
- **Exécuter** ensuite votre fonction ***run_models*** sur vos données et interpréter les résultats obtenus.
- **Améliorer** la vectorisation proposée par la technique **Tf–idf**. Exécuter ensuite à nouveau votre fonction ***run_models*** sur vos données et interpréter les résultats obtenus en les comparant à ceux obtenus à l'étape précédente.
- **Proposer** une approche de **sélection de mots clés pertinents** (***sélection de variables***).
- **Appliquer** la méthode **SVD** de **réduction de dimensions** (**TruncatedSVD**) afin de construire des "**concepts**" liés aux documents et aux termes. Elle permettra entre autres de résoudre les problèmes de **synonymie** (plusieurs mots avec un seul sens) et de **polysémie** (un seul mot avec plusieurs sens). La fonction suivante vous aidera à comprendre les concepts en affichant leurs mots les plus pertinents.

```
def print_top_words(model, feature_names, n_top_words):
    for topic_idx, topic in enumerate(model.components_):
        message = "Concept #%d: " % topic_idx
        message += " ".join([feature_names[i] for i in topic.argsort()[:-n_top_words - 1:-1]])
        print(message)
    print()
```

```
print_top_words(SVD, vocabulaire(), 10) # pour afficher les 10 mots les plus pertinents par concept
```

- **Exécuter** ensuite votre fonction ***run_models*** sur vos données et interpréter les résultats obtenus. Vous pouvez toujours appliquer votre fonction sur les **données pré-traitées** et les **données non pré-traitées** afin d'analyser l'apport de la partie pré-traitement de données.

A partir de ce point il faut travailler soit avec ou sans les données pré-traitées. N'oubliez pas qu'un minimum traitement est nécessaire.

- **Dans un nouveau notebook (inspiré de celui sur moodle Word2Vec_creation.ipynb)**, **proposer** un code qui permettra d'apprendre votre propre modèle de plongement lexical **Word2Vec** sur vos données textuelles. **Évaluer visuellement et numériquement** sur quelques mots clés votre nouveau modèle de vectorisation (*Embedding*).
- Exploiter votre modèle **Word2Vec** pour la vectorisation de vos textes (avec deux méthodes utilisant ou non le TF-IDF des mots). Exécuter ensuite à nouveau votre fonction ***run_models*** sur vos données vectorisées par **Word2Vec** et interpréter les résultats obtenus en les comparant à ceux obtenus aux étapes précédentes. Vous pouvez vous inspirer de la fonction suivante :

```
## get word2vec for each sentence by using average word embeddings
def word2vec_generator(texts,model,vector_size):
    dict_word2vec = {}
    for index, word_list in enumerate(texts):
        arr = np.array([0.0 for i in range(0, vector_size)])
        nb_word=0
        for word in word_list:
            try:
                arr += model[word]
                nb_word=nb_word+1
            except KeyError:
                continue
        if(len(word_list) == 0):
            dict_word2vec[index] = arr
        else:
            dict_word2vec[index] = arr / nb_word
```

df_word2vec = pd.DataFrame(dict_word2vec).T
return df_word2vec

- Idem en utilisant le **modèle Word2Vec pré-entrainé de Google ou un autre**.
- ***Pipeline* :** Automatiser l'enchainement de votre meilleur traitement dans un pipeline

2. Partie 2 : *Analyse de données d'articles médicaux*

L'objectif dans cette partie est d'adapter puis d'appliquer votre code précédemment développé sur un jeu de données *d'articles* médicaux du fichier **"PubMed-multi-label-dataset.csv"**. Chaque texte dans ce jeu de données est labélisée par un ensemble de labels parmi les 14 labels suivants : Anatomy [**A**], Organisms [**B**], Diseases [**C**], Chemicals and Drugs [**D**], Analytical, Diagnostic and Therapeutic Techniques, and Equipment [**E**], Psychiatry and Psychology [**F**],Phenomena and Processes [**G**], Disciplines and Occupations [**H**], Anthropology, Education, Sociology, and Social Phenomena [**I**], Technology, Industry, and Agriculture [**J**], Information Science [**L**], Named Groups [**M**], Health Care [**N**], Geographicals [**Z**]. **Le résumé (Abstract)** sera considéré pour la représentation textuelle de chaque article.

Vous devez remplacer la fonction ***run_models*** pour comparer plusieurs modèles d'apprentissage multi-label. Vous utiliserez l'approche **EnsembleClassifierChain** et **MultiOutputCLassifier** du package **sklearn.multioutput**. Ces classifieurs nécessite un classifieur de base (**base_estimator**) dont vous avez le libre choix d'utilisation. Votre évaluation se basera sur les mesures **hamming_loss**, **zero_one_loss, Macro F1 et Micro F1**.